Guía del alumno · S10 RAG con PostgreSQL y modelos reales

La práctica autónoma dura aproximadamente **4–6 horas**, con Docker y acceso a los modelos preparados. Sigue esta guía desde la raíz de tu clon. No necesitas interfaz web ni despliegue. El guion del profesor y los resultados de preparación están en `material/S10/`; no sustituyen tu evaluación con tu propio corpus.

1. Comprobar el entorno

Necesitas Git, Bun y Docker. Descarga el repositorio del curso y abre una terminal en su raíz, donde está `package.json`.

```
bun install --frozen-lockfile
cp lab/.env.example lab/.env
```

Abre `lab/.env` con tu editor y añade tus claves privadas. La configuración ensayada usa `ANTHROPIC_API_KEY` para Sonnet y `OPENROUTER_API_KEY` para Gemini Embedding 2. Nunca compartas ese archivo ni sus valores. Sus variables tienen prioridad para este laboratorio.

El enunciado propone Gemini como referencia de generación. Si lo usas, configura explícitamente proveedor, modelo, clave y tarifas de reserva. Registra los modelos realmente ejecutados: cambiar una etiqueta no cambia el modelo.

```
bun run lab:real preflight
```

Esperado: backend `postgresql-pgvector`, dimensión 768, identidades de los modelos y `missing: []`. Esta comprobación no llama a APIs. Si faltan credenciales, corrígelo localmente; no las envíes a una IA ni al profesor.

2. Arrancar PostgreSQL con pgvector

```
docker compose -f lab/docker-compose.yml up -d
bun run lab:real status
```

La base es exclusiva de la práctica y escucha solo en `127.0.0.1:54329`. Al principio puede tener cero fragmentos; deben existir la extensión pgvector y el índice HNSW. Si Docker no está listo, espera a su comprobación de salud y vuelve a leer `status`. No cambies el host para apuntar a una base compartida: el CLI lo rechaza.

Abre `lab/schema.sql`. Identifica `vector(768)`, `vector_cosine_ops` e HNSW. En este índice `<=>` es distancia coseno (menor es más próximo); la similitud es `1 - distancia`.

3. Leer el corpus y reconstruir

Desde el 24 de octubre de 2026 la referencia contiene 12 políticas de **Zocomar**, una tienda en línea inventada (devoluciones, electrónica y garantía, envíos vigentes y una política de 2025 ya sustituida,

pagos, vendedores, disputas, Plus, productos restringidos, atención al cliente y promociones). Los 10 documentos de AetherCloud usados en S10 se conservan en `lab/corpus/aethercloud-s10/`. Tu entrega necesita al menos ocho documentos ficticios de un dominio coherente. Conserva títulos, secciones e IDs. No incluyas datos personales, documentos internos ni claves.

Antes de consumir APIs, el titular de la cuenta debe autorizar el gasto. Los comandos siguientes usan el tope con el que se ensayó Zocomar: **5 USD acumulativos en reservas conservadoras**, de los que RAG y CRAG completos reservaron 2,77 USD. El tope debe ser el mismo en todos los comandos del mismo diario. No es una tarifa fija ni una factura; usar otra cuenta o modelo exige verificar sus condiciones.

```
bun run lab:real rebuild --execute --max-spend-usd 5
bun run lab:real status
```

Esperado con el corpus de referencia: 12 documentos, 54 fragmentos, dimensión mínima y máxima 768 y definición HNSW. La ingesta obtiene embeddings reales antes de reemplazar el índice y usa una transacción. El estado de Zocomar vive en `lab/.local/zocomar/`; el recibo queda en `lab/.local/zocomar/index.json`.

Lee `lab/src/corpus.ts` y `lab/src/pgvector-store.ts`. Explica qué se limpia, por qué se fragmenta y qué se conserva como procedencia. Consulta `lab/.local/zocomar/index.json` para identificar el proveedor, el modelo y la versión de corpus, sin claves.

4. Calibrar con validación

```
bun run lab:real calibrate --execute --max-spend-usd 5
```

Se usan seis preguntas de validación, separadas del test. Con Zocomar la configuración ensayada produjo `0,722598` (con AetherCloud, en S10, `0,724767`); tu resultado puede ser diferente. Si hay solapamiento entre positivos y negativos, el comando se detiene: investiga con validación, sin abrir el test para elegir el corte.

El archivo `lab/.local/zocomar/calibration.json` registra las puntuaciones y la identidad del índice. Cambiar corpus, proveedor o embedding exige reconstruir y recalibrar. No copies `0,25` del respaldo offline ni `0,50` de la web del curso.

5. Consultar, abrir la fuente y comprobar un rechazo

```
bun run lab:real query --question '¿Cuántos días tiene el vendedor para entregar el
paquete al transportista?' --execute --max-spend-usd 5 --brief
```

La respuesta está en la política de vendedores (2 días laborables, 1 para los artículos con «Exprés disponible»). Abre el Markdown original y comprueba cada afirmación. No basta con que la cita tenga buen formato.

```
bun run lab:real query --question '¿Cuál es el teléfono personal del director de
Zocomar?' --execute --max-spend-usd 5 --brief
```

El dato no consta. Comprueba qué hace tu sistema: si ningún vecino supera el corte, rechaza con `generationCalled: false`; si alguno lo supera por poco, el generador recibe contexto que no responde. Esta consulta sí usa un embedding. El recibo completo de cada consulta se guarda en `lab/.local/zocomar/last-query.json`; copia los que quieras conservar antes de otra consulta.

En el código, sigue: `queryNearest` → candidatos → `selected` → rechazo o generador → filtro de citas. El adaptador real está en `apps/api/src/lab-models.ts` y reutiliza AI SDK y la fábrica del curso.

6. Congelar y ejecutar la evaluación

En tu entrega diseña **20 preguntas nuevas**, 14 respondibles y seis sin respuesta. Escribe antes la respuesta esperada y el documento fuente, o el motivo del rechazo. No indexes preguntas ni respuestas esperadas. Usa otras preguntas para elegir prompt, fragmentación y umbral.

El conjunto de referencia de Zocomar es `LAB_TEST_SET_20` en `lab/src/evaluation.ts`, separado de la validación y de los casos de demostración. `eval` lo ejecuta una vez en RAG básico y en CRAG:

```
bun run lab:real eval --execute --max-spend-usd 5
```

Conserva `lab/.local/zocomar/evaluation.json`. El CLI bloquea una nueva sobrescritura automática. Revisa manualmente los 20 casos: respuesta completa, cifras, fuentes y rechazo. Informa **X/14** respondibles y **Y/6** rechazos correctos. Conserva los fallos; no se exige que el sistema acierte todo.

El ensayo de S10 con AetherCloud (`lab/eval/s10-real-test-20.json`) obtuvo **9/14 y 6/6**, con tres abstenciones incorrectas y dos respuestas parciales. Con Zocomar, el 24 de octubre, RAG básico y CRAG obtuvieron **14/14 y 6/6** los dos. Ninguno se reajustó después ni es una tasa general de calidad.

7. S11: CRAG con modelos reales

El grafo de `lab/src/crag-graph.ts` usa en `lab:real` un juez real (`apps/api/src/lab-models.ts`): puntúa cada candidato en `[0, 1]`, estima la cobertura de la pregunta y nombra lo que falta; los umbrales `0,50 / 0,40 / 0,75` que deciden la ruta están en código (`decideLabSufficiency`). RAG y CRAG comparten índice, embeddings y generador.

```
bun run lab:real compare --question 'Devolví en un Zocopunto un pedido que pagué con
Bizum. ¿Cuánto puede tardar en llegarme el dinero?' --execute --max-spend-usd 5 --brief
bun run lab:real demo --execute --max-spend-usd 5
```

`compare` ejecuta la misma pregunta en las dos etapas e imprime la similitud de cada vecino junto a la relevancia del juez, las consultas, las rutas y las dos respuestas. `demo` recorre los seis casos visibles de `LAB_DEMO_CASES`. Además del acierto, mira el **margen** (cuánto separa cada señal las preguntas con respuesta de las que no la tienen) y **qué llega al generador**: en el ensayo, RAG pasó la política de envíos sustituida en 3 de 14 respuestas y CRAG en ninguna. Evidencia: `docs/evidence/zocomar-lab-2026-10-24.md`.

8. Preparar la entrega

Incluye código de ingesta y consulta; ocho o más documentos ficticios; configuración de PostgreSQL/pgvector y modelos sin secretos; expectativas y resultados de las 20 preguntas; y un README con los comandos, versiones, escala de puntuación y criterio de corte. Registra candidatos,

puntuaciones, respuesta, citas, latencia y uso cuando se devuelva. Un coste desconocido se declara desconocido.

La ampliación con PDFs o interfaz es opcional. Para S10 no hace falta desarrollar el reranking ni la corrección de S11.

Si necesitas ayuda

- `missing` en preflight: falta una clave o configuración; compruébala sin imprimirla.
- Conexión rechazada en status: revisa Docker y el puerto local. No apuntes a una base ajena.
- Error 401/403 del proveedor: revisa la cuenta y sus permisos. No pegues tokens.
- Presupuesto insuficiente: detente y consulta al titular. No borres el diario ni aumentes el tope por tu cuenta.
- Error de dimensión: usa el mismo modelo y 768 dimensiones en documentos y preguntas; no rellenes ni recortes el vector manualmente.
- API fallida o timeout: conserva el recibo y la reserva; no repitas automáticamente un envío incierto.
- `running.lock`: verifica que no haya otro proceso antes de intervenir.

Para pedir ayuda a una IA, envía el comando **sin claves**, el mensaje de error saneado, las versiones y si estás en modo real u offline. Puedes pedir: «Explica en qué etapa falla esta consulta, qué evidencia falta y qué archivo debo revisar; no cambies el umbral con las preguntas del test».

El respaldo `bun run lab:demo` funciona sin modelos y se identifica como determinista. Es útil para estudiar el flujo; no acredita la ejecución real exigida por la práctica.